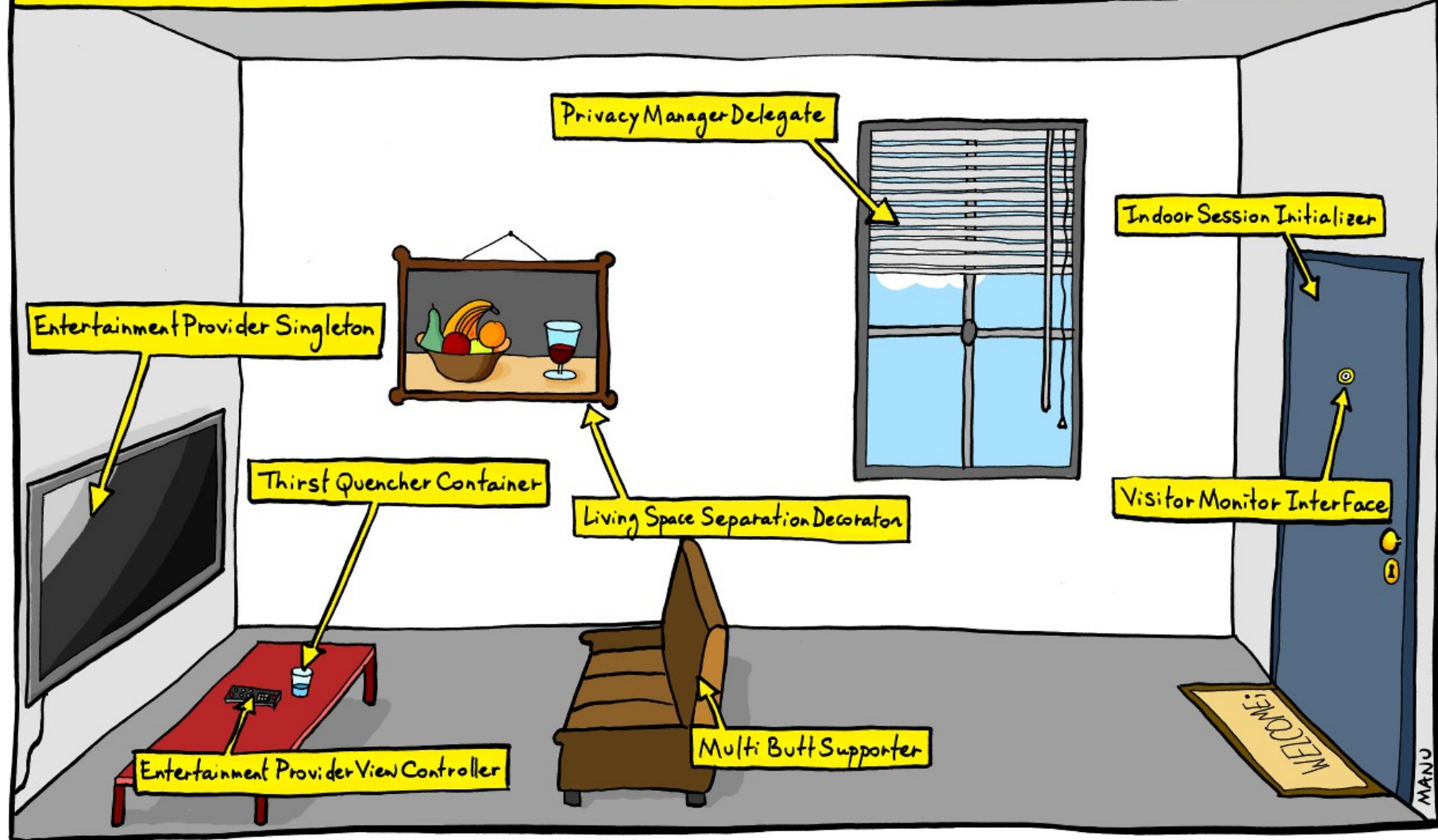




Java™

THE WORLD SEEN BY AN "OBJECT-ORIENTED" PROGRAMMER.





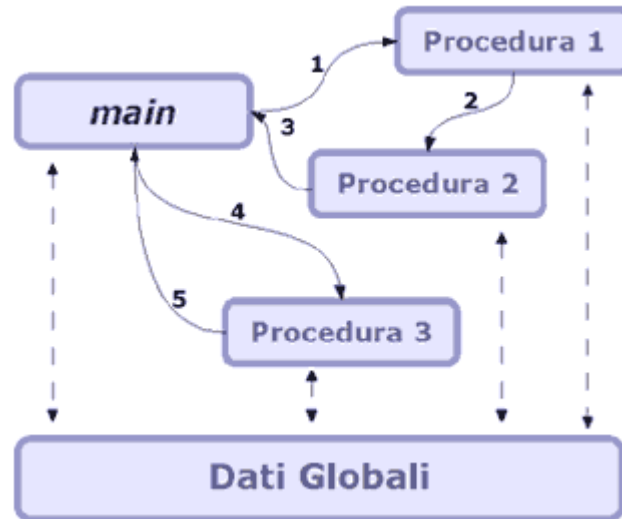
PROGRAMMAZIONE NON STRUTTURATA

LIST

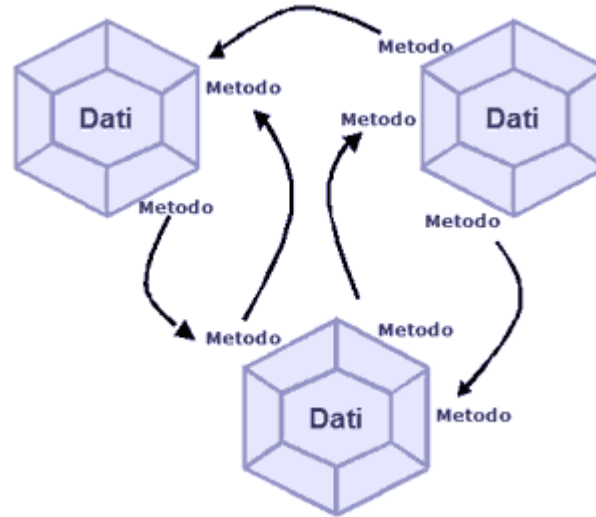
```
10 PRINT "THIS IS POISSON, BY L.F.GOELLER, 11/26/78."
20 PRINT "P IS PROBABILITY OF EXACTLY N TRUNKS BUSY."
30 PRINT "P1 IS POISSON BLOCKING PROBABILITY."
40 INPUT "TRAFFIC IN ERLANGS OFFERED GROUP =",A: LET E=EXP(A)
50 LET N=0: LET T=1: LET T1=1: LET T2=0: LET Z=10
60 LET P1=1-T2: LET T2=T1/E: LET P=T/E
70 IF P<.0001 THEN 110
80 PRINT "N= ";%4I;N;"      P = ";%6F5;P;"      P1=";%7F5;P1
90 IF N<Z THEN 120:
91 STOP : LET Z=Z+10
100 PRINT "OFFERED TRAFFIC WAS ",A
110 IF N<Z THEN 120: LET Z=Z+10
120 LET N=N+1: LET T=T*A/N: LET T1=T1+T
130 IF P1>.0001 THEN 60
140 END
```

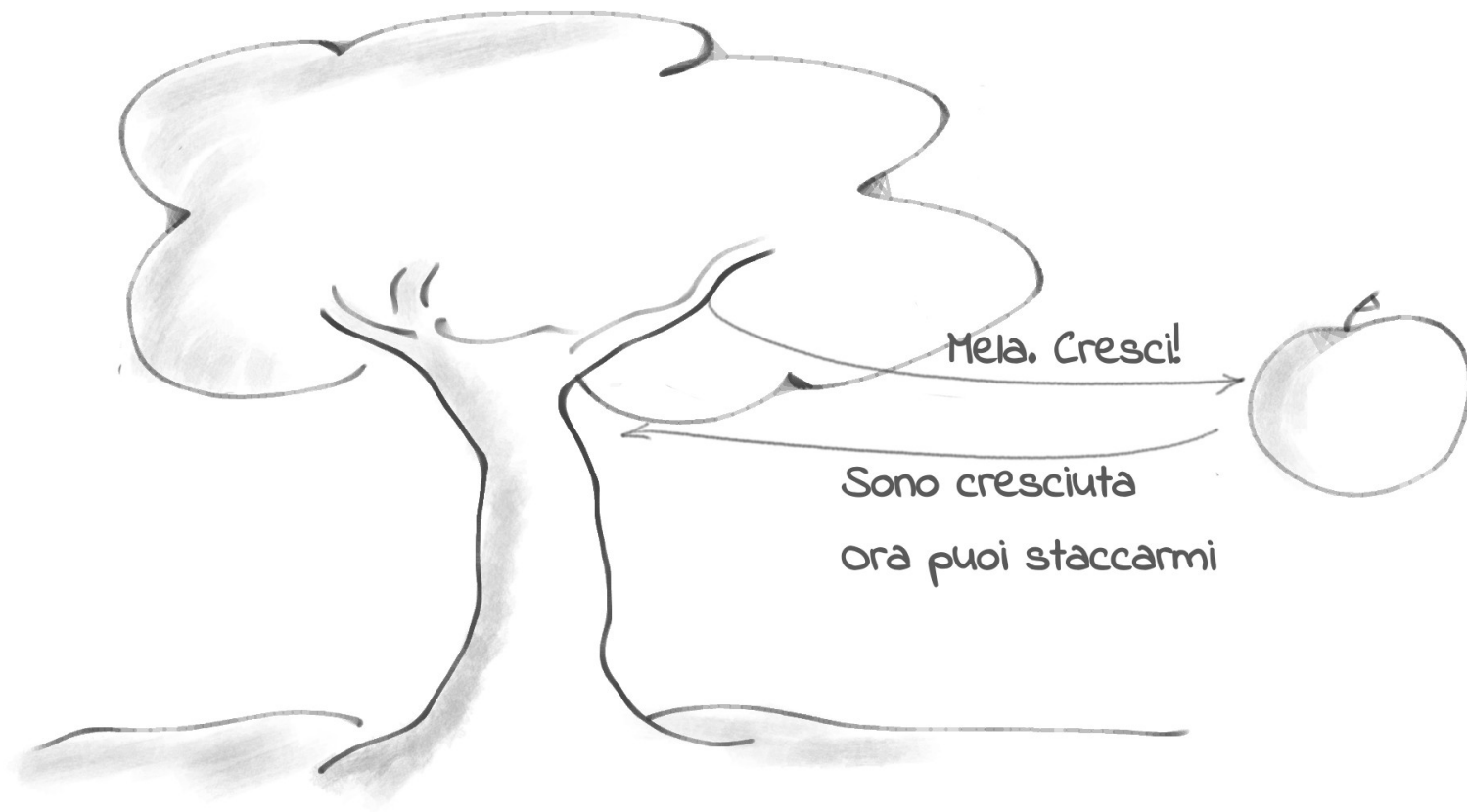
Figure 6. Program listing for Poisson.

PROGRAMMAZIONE PROCEDURALE



PROGRAMMAZIONE A OGGETTI



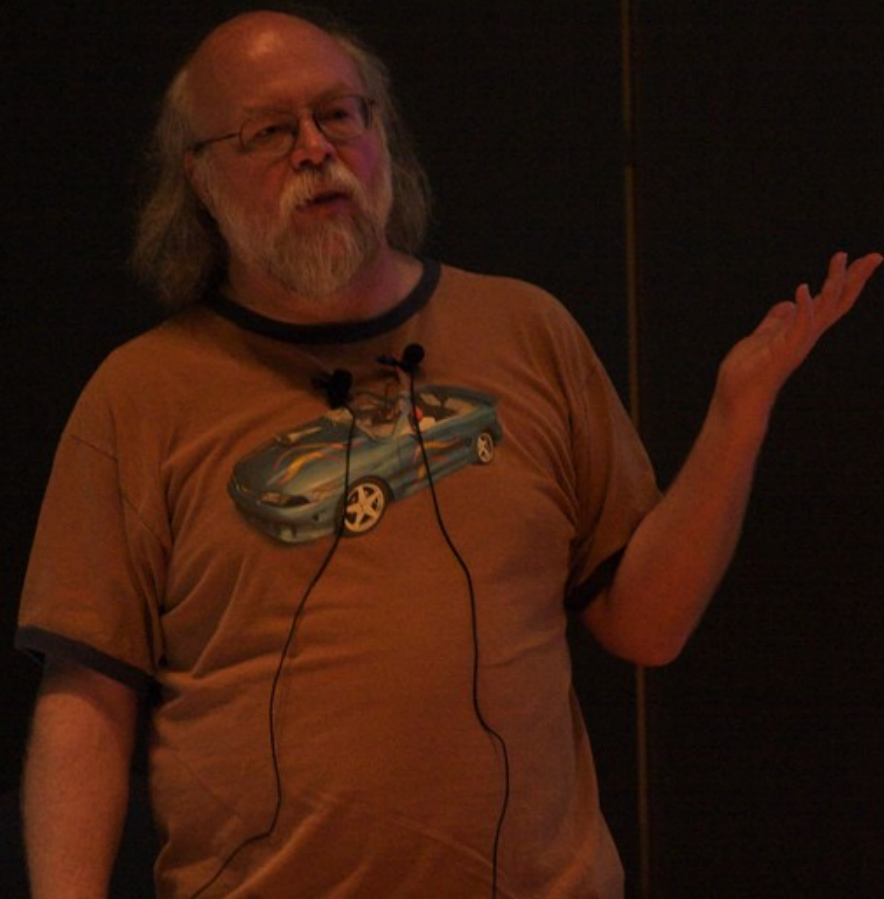


Mela. Cresci!

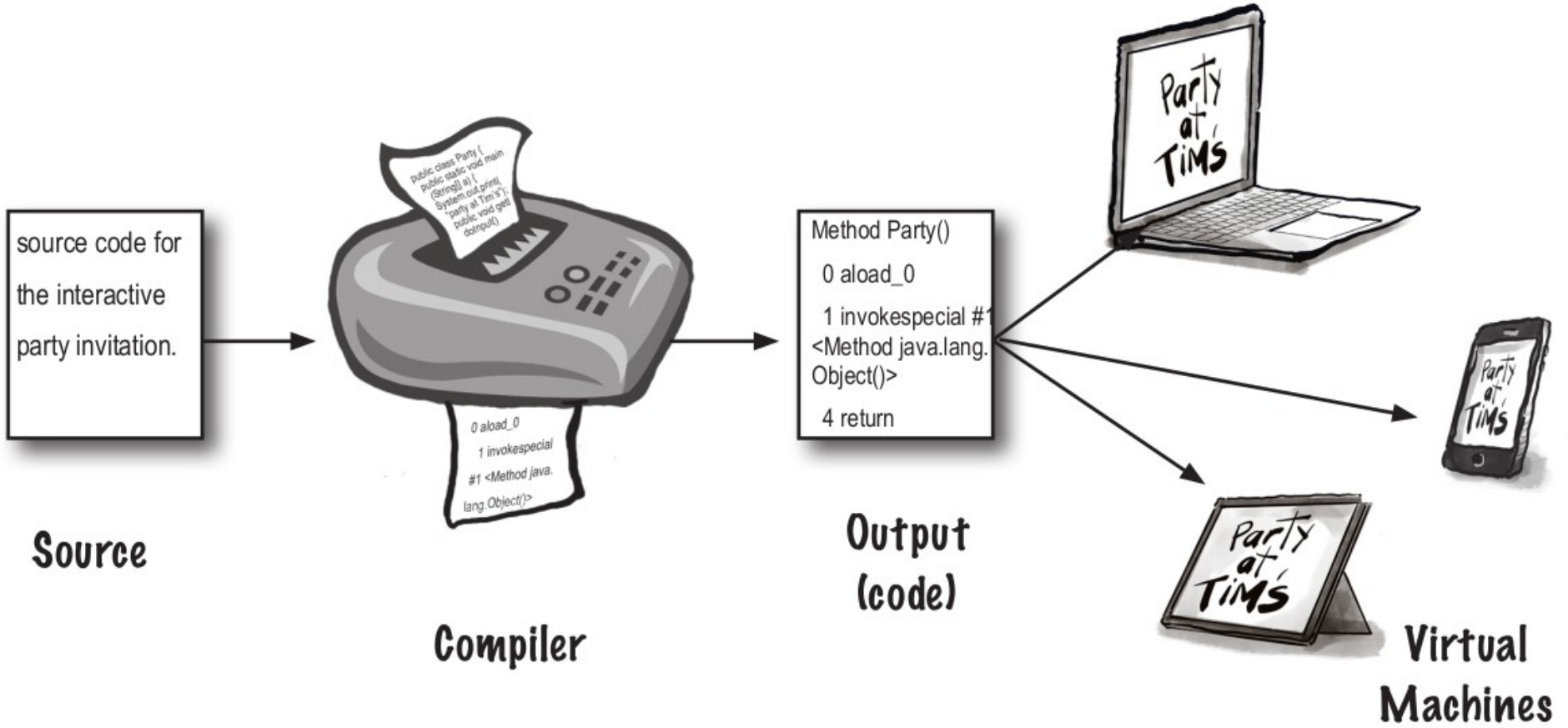
Sono cresciuta
ora puoi staccarmi



Ole-Johan Dahl e Kristen Nygaard, Simula I e Simula 67







```
import java.awt.*;
import java.awt.event.*;

class Party {
    public void buildInvite() {
        Frame f = new Frame();
        Label l = new Label("Party at Tim's");
        Button b = new Button("You bet");
        Button c = new Button("Shoot me");
        Panel p = new Panel();
        p.add(l);
    } // more code here...
}
```

Source

```
File Edit Window Help Plead
%javac Party.java
```

Compiler

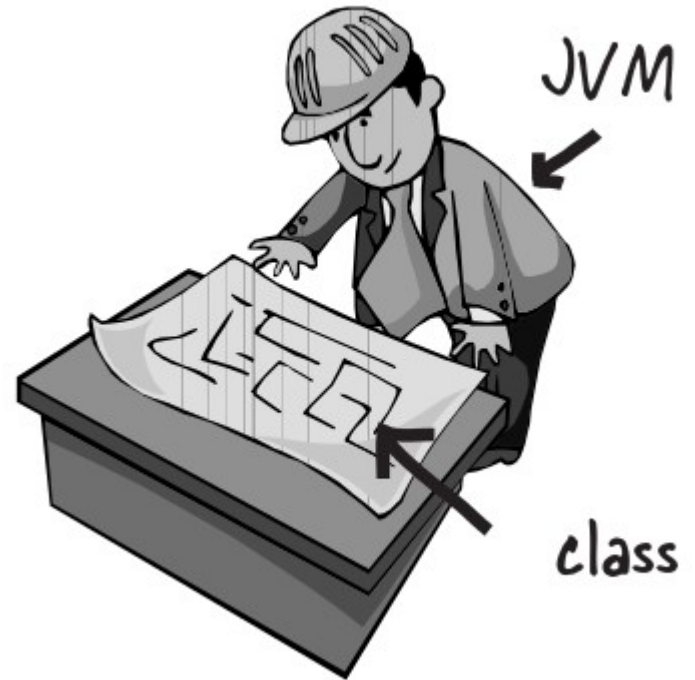
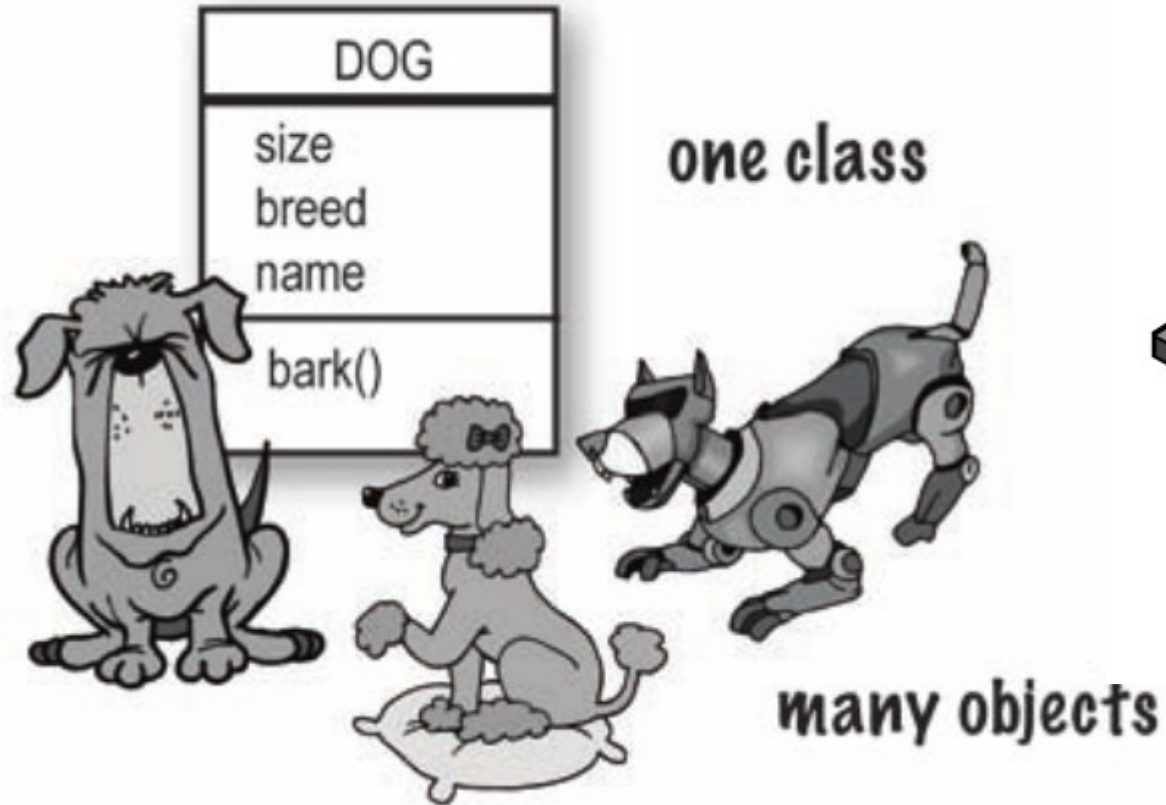
```
Method Party()
  0 aload_0
  1 invokespecial #1 <Method
  java.lang.Object()>
  4 return
Method void buildInvite()
  0 new #2 <Class java.awt.Frame>
  3 dup
  4 invokespecial #3 <Method
  java.awt.Frame()>
```

**Output
(code)**



**Virtual
Machines**

LA CLASSE



source file

class definition

method 1

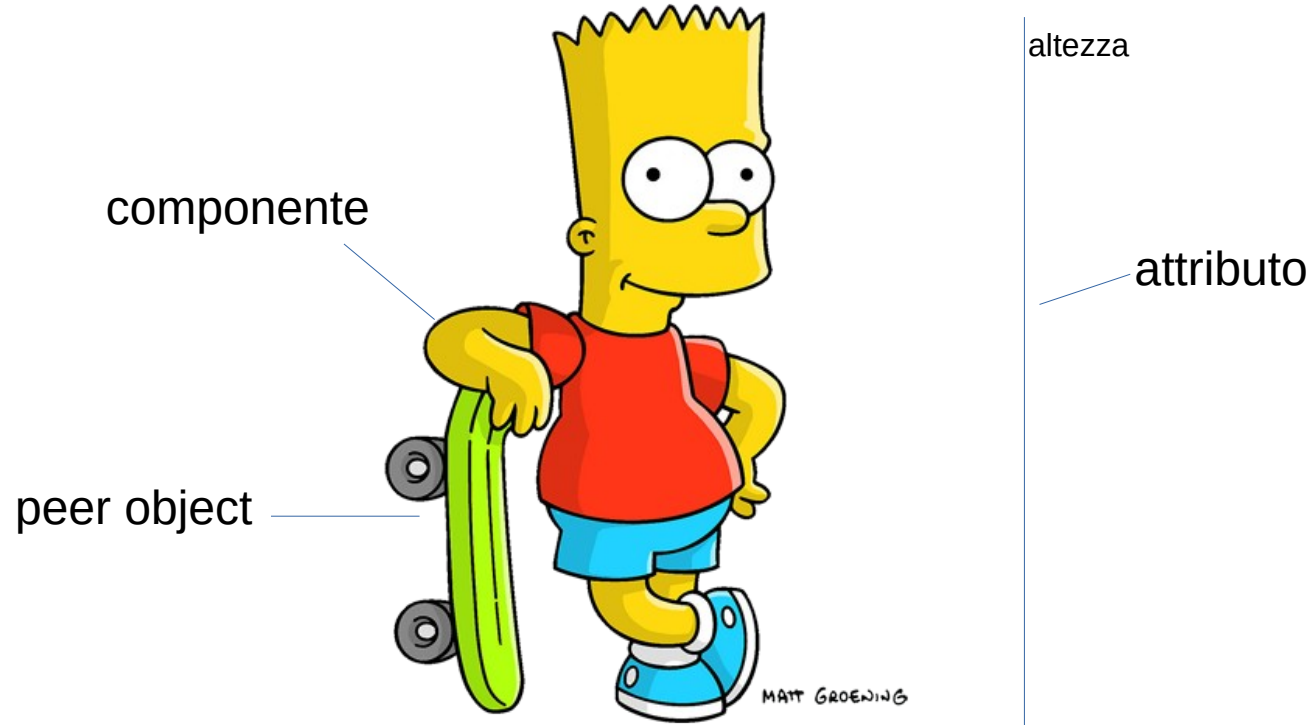
statement

method 2

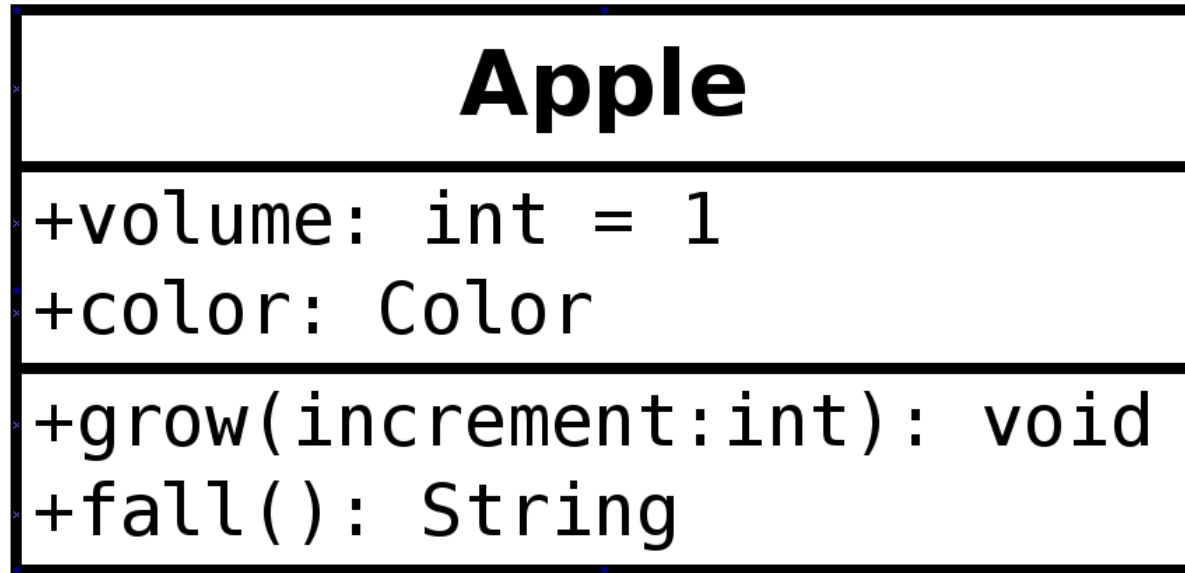
statement

statement

TIPOLOGIE DI PROPRIETÀ



SCHEMA UML DI UNA CLASSE JAVA

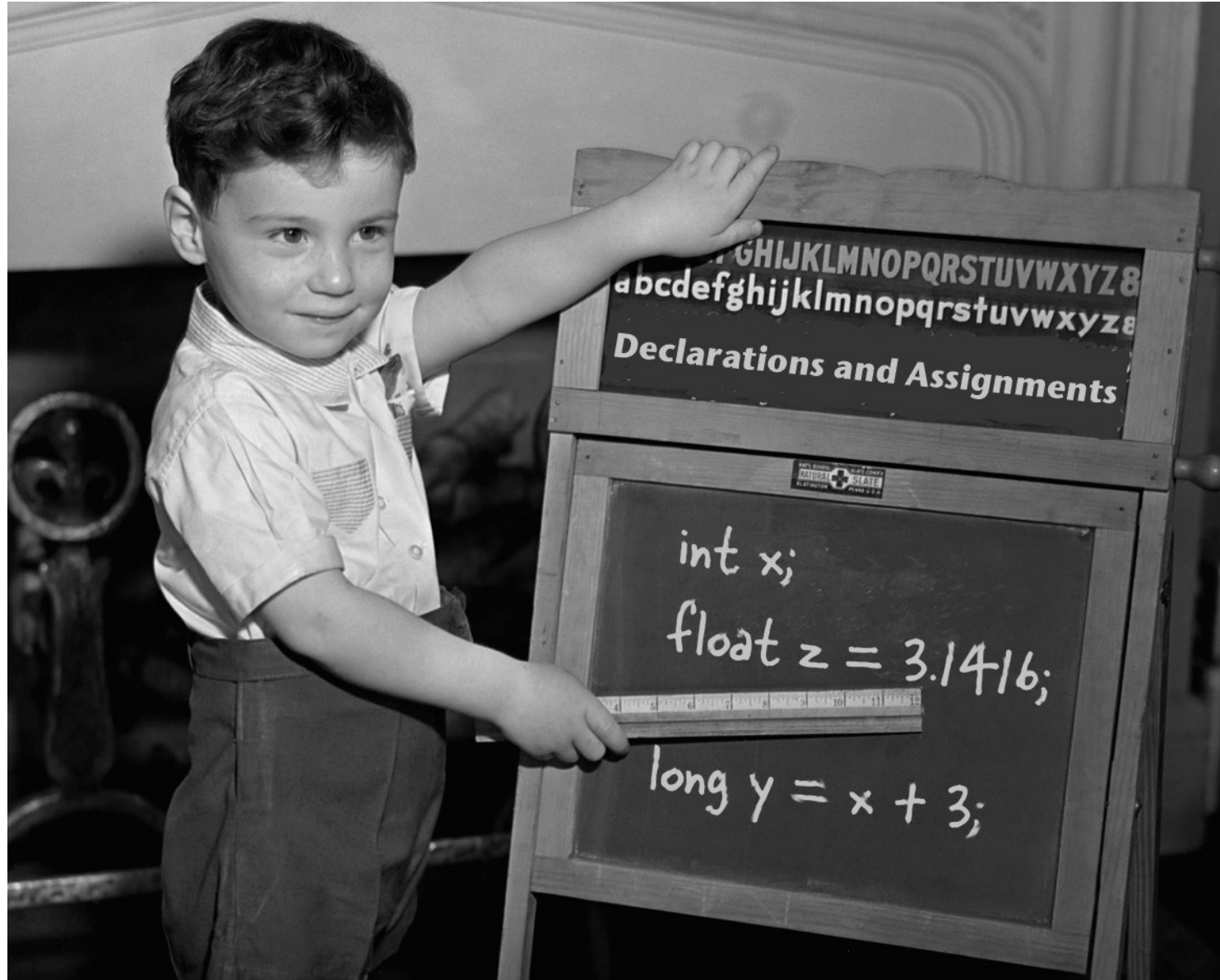


CODICE DI UNA CLASSE JAVA

```
public class Apple {
```

```
// qui mettiamo il codice della classe mela
```

```
}
```



A B C D E F G H I J K L M N O P Q R S T U V W X Y Z &
a b c d e f g h i j k l m n o p q r s t u v w x y z &

Declarations and Assignments

int x;

float z = 3.1416;

long y = x + 3;

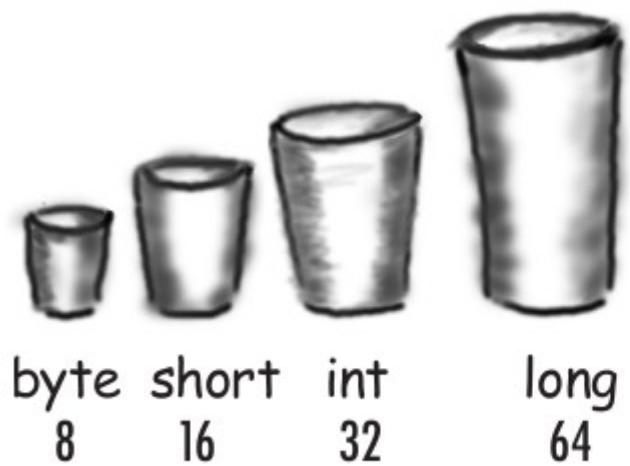


`int count;`

type ↗ ↖ name

CODICE DI UNA CLASSE JAVA

```
public class Apple {  
    int volume = 0;  
  
}
```



Primitive Types

Type	Bit Depth	Value Range
------	-----------	-------------

boolean and char

boolean	(JVM-specific)	true or false
---------	----------------	-----------------------------

char	16 bits	0 to 65535
------	---------	------------

numeric (all are signed)

integer

byte	8 bits	-128 to 127
------	--------	-------------

short	16 bits	-32768 to 32767
-------	---------	-----------------

int	32 bits	-2147483648 to 2147483647
-----	---------	---------------------------

long	64 bits	-huge to huge
------	---------	---------------

floating point

float	32 bits	varies
-------	---------	--------

double	64 bits	varies
--------	---------	--------

Primitive declarations with assignments:

```
int x;  
x = 234;  
byte b = 89;  
boolean isFun = true;  
double d = 3456.98;  
char c = 'f';  
int z = x;  
boolean isPunkRock;  
isPunkRock = false;  
boolean powerOn;  
powerOn = isFun;  
long big = 3456789L;  
float f = 32.5f;
```

Note the 'f' and 'L'. With some number types, you have to specifically tell the compiler what you mean, or it might get confused between similar-looking number types. You can use upper or lowercase.



```
thisIsNotAJoke;  
_myVariable;  
$importantVar;
```

Do this... ↗

Begins with number, not good. →

Begin with symbols (% and ~) that aren't allowed. →

```
3zip;
```

```
%entage;
```

```
~approx;
```

↖ ...not this.



```
my3sons;  
cost$;  
vitaminB12;
```

Do this... ↗

Got a space, not allowed ↘

Got -, + signs. Not allowed and will seriously confuse JavaScript. →

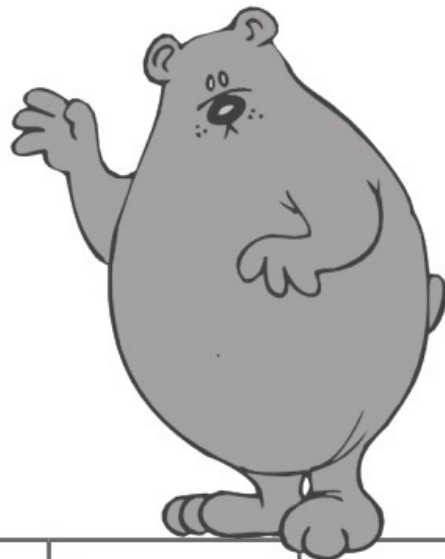
```
zip code;
```

```
first-name;
```

```
to+do;
```

↖ ...not this.





This table reserved

_	catch	double	float	int	private	super	true
abstract	char	else	for	interface	protected	switch	try
assert	class	enum	goto	long	public	synchronized	void
boolean	const	extends	if	native	return	this	volatile
break	continue	false	implements	new	short	throw	while
byte	default	final	import	null	static	throws	
case	do	finally	instanceof	package	strictfp	transient	

CODICE DI UNA CLASSE JAVA

```
public class Apple {  
    int volume = 0;  
  
    public void grow(int increment) {  
        // implementiamo il metodo grow  
    }  
  
    public String fall() {  
        // implementiamo il metodo fall  
    }  
}
```

OPERATORI MATEMATICI E DI ASSEGNAZIONE

// dichiarazione

int a;

// assegnazione

a = 1;

int b = 1;

// somma e concatenazione

a = b + 1;

a += b // a = a + b

String s = "la somma di a e b è uguale a " + (a + b);

// moltiplicazione, divisione e sottrazione

a = b * 5;

a = b / 6;

a = b - 1;

a *= b; // a = a * b

// resto di una divisione

a = 44 % 6;

++

```
int a = 1;
```

```
int b = 2;
```

```
a++ // a = a + 1 = 2
```

```
++a // a = a + 1 = 3
```

```
a = b++; // a = 2, b = 3
```

```
a = ++b; // a = 4, b = 4
```

OPERATORI DI CONFRONTO

`==` `!=` `>` `>=` `<` `<=`

```
int a = 1;
```

```
int b = 2;
```

```
boolean response = a == b; // a è uguale a b. Vero o falso?
```

```
// response is false
```

```
boolean response = b > a; // b è maggiore di a. Vero o falso?
```

```
// response is true
```



Exercise

Valutare le seguenti espressioni valorizzando le eventuali variabili con i valori sul lato destro.

$(9 / 5) * \text{tempC} + 32$ _____ (tempC = 10)

"Number" + " " + "2" _____

level >= 5 _____ (level = 5, level = 10)

color != "rosa" _____ (color = "blu")

"La somma di $x + 2$ è pari a " + $x + 2$ _____ (x = 3)



↖
Not this kind of expression!



Exercise

Valutare le seguenti espressioni valorizzando le eventuali variabili con i valori sul lato destro.

$(9 / 5) * \text{tempC} + 32$ 50 ($\text{tempC} = 10$)

"Number" + " " + "2" Number 2

$\text{level} \geq 5$ true ($\text{level} = 5, \text{level} = 10$)

$\text{color} \neq \text{"rosa"}$ true ($\text{color} = \text{"blu"}$)

"La somma di $x + 2$ è pari a " + $x + 2$ 32 ($x = 3$)



Not this kind of expression!

CODICE DI UNA CLASSE JAVA

```
public class Apple {  
    int volume = 0;  
  
    public void grow(int increment) {  
        volume += increment;  
    }  
  
    public String fall() {  
        return "Cadooo!";  
    }  
}
```

CREAZIONE E USO DI UN OGGETTO

// creo una nuova mela

Apple apple = new Apple();

// assegno il volume della mela a una variabile

int volume = apple.volume;

// chiedo alla mela di fare certe cose

apple.grow(10);

String appleMessage = apple.fall();

Public so everyone can access it

This is a class (duh)

The name of this class

Opening curly brace of the class

```
public class MyFirstApp {
```

(We'll cover this one later.)

The return type. void means there's no return value.

The name of this method

Arguments to the method. This method must be given an array of Strings, and the array will be called 'args'

Opening brace of the method

```
public static void main (String[] args) {
```

```
System.out.print("I Rule!");
```

Every statement MUST end in a semicolon!!

This says print to standard output (defaults to command line)

The String you want to print

```
}
```

Closing brace of the main method

```
}
```

Closing brace of the MyFirstApp class

UNNAMED (IMPLICIT) CLASS

```
void main() {  
    IO.println("I Rule!");  
}
```

New in Java 25!

&& // AND || // OR ^ //XOR ! // NOT

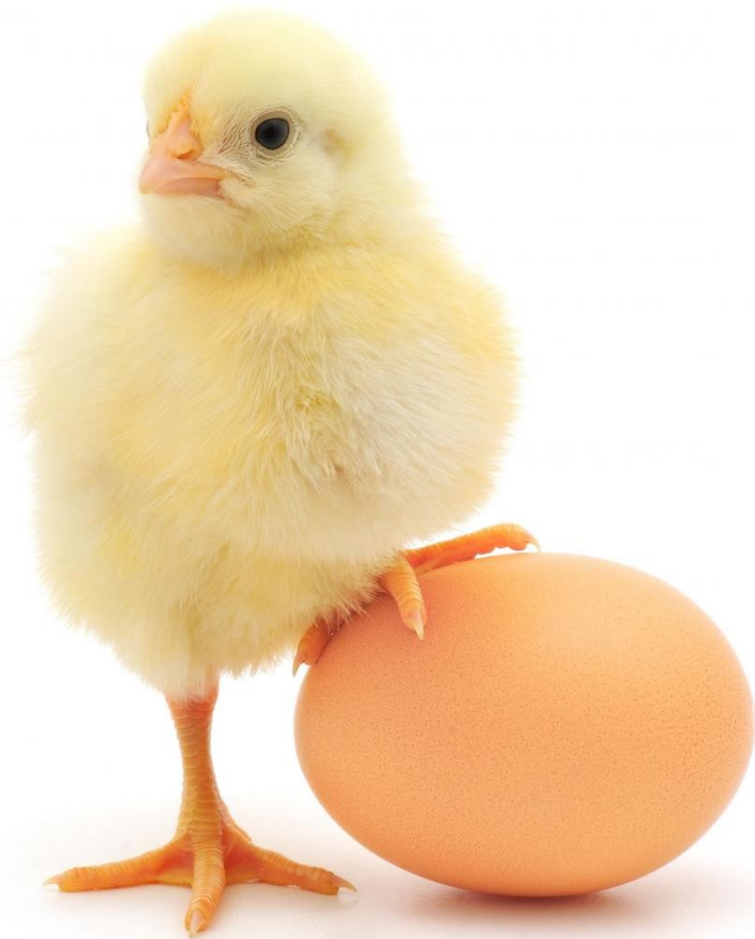
a == 1 && b > 2 // true se entrambe le espressioni sono vere
// true solo se a = 1 e b > 2

c > 0 || d == 0 // true se almeno un'espressione è vera
// true se c > 0 oppure d = 0

e > 0 && f == 1 || g < 5
// true se (e > 0 e f = 1) oppure g < 5

h == 1 ^ i == 10 // true se solo una delle due espressioni è vera
// true se h != 1 e i == 10 oppure se h == 1 e i != 10

!m // negazione
// true se m è false e false se m è true



Abbiamo 86 centesimi per comprare le uova che costano 9 centesimi l'una.

Quante uova possiamo acquistare?

```
double budget = 0.86;  
double eggPrice = 0.09;  
int eggNumber;
```



```
int x = 24;  
byte b = x;  
//won't work!!
```



ARIANE 5

FIRST FLIGHT FAILURE

V501 - JUNE 4, 1996

COSTANTI

```
public static final int MAX_SPEED = 65;
```

Esempi:

```
double area = r * r * Math.PI;  
Color color = Color.RED;
```



CONDIZIONI

```
if (condizione) {  
    // esegue se la condizione è = true  
} else {  
    // esegue se la condizione è = false  
}
```

Esempio:

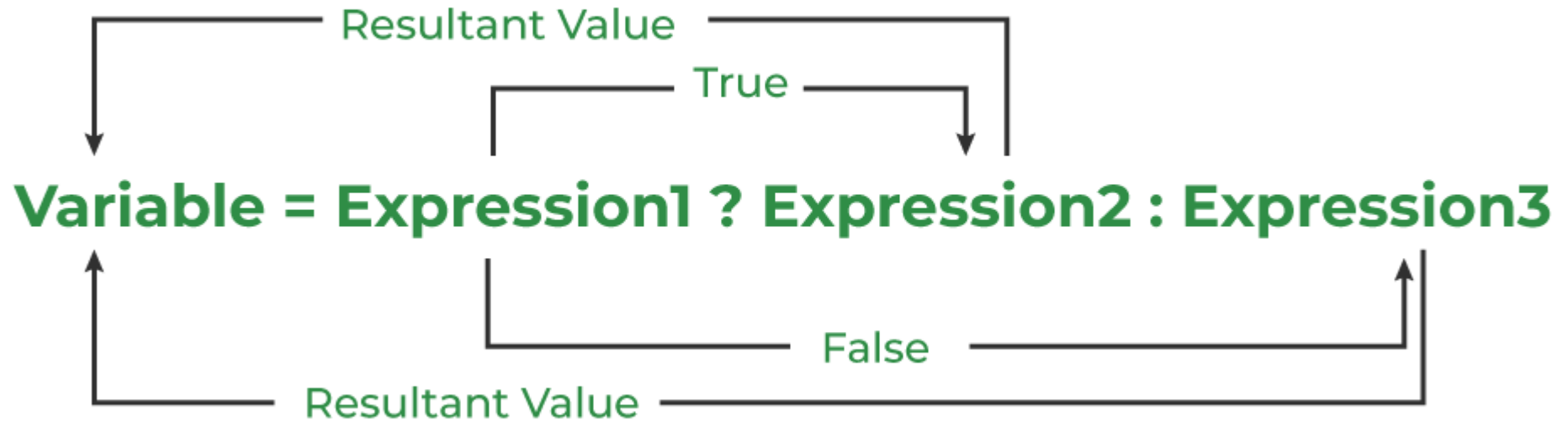
```
if (scoops < 4)  
    System.out.print("Il gelato sta finendo");  
else  
    System.out.print("Veloce che il gelato si scioglie!");
```



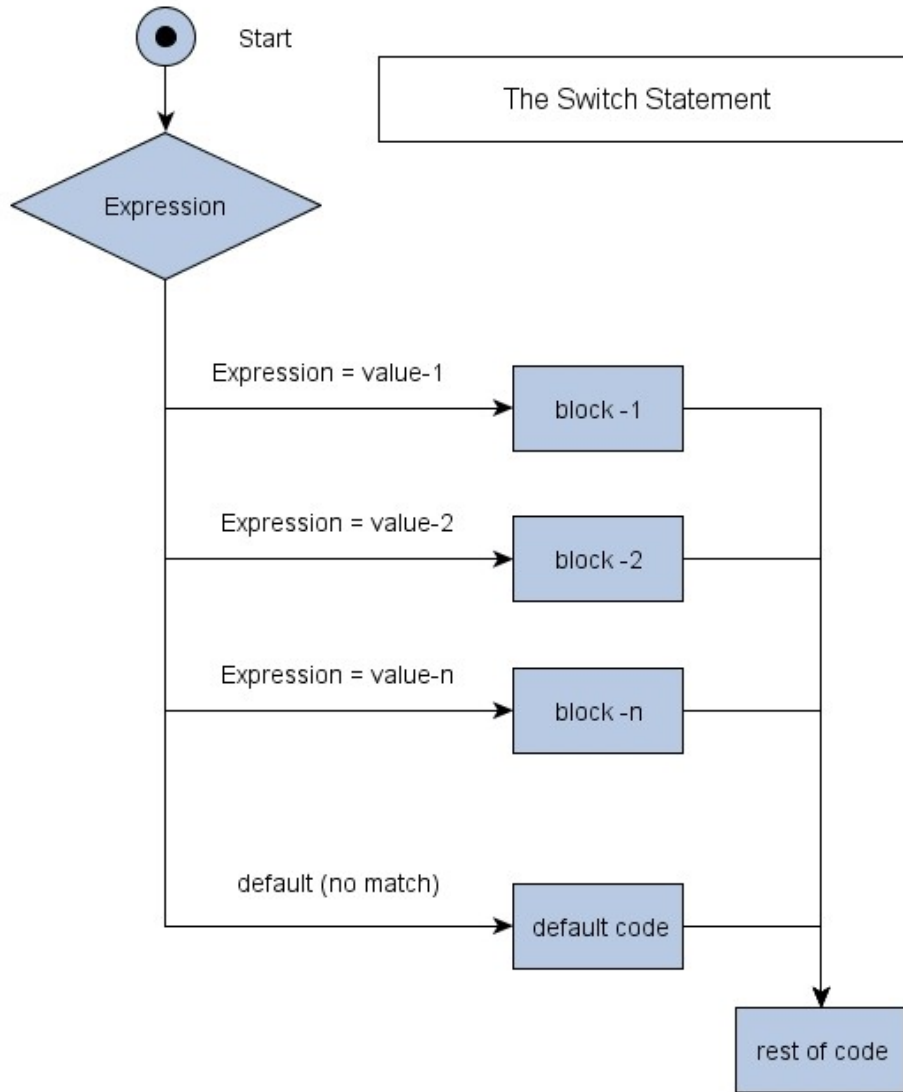
```
if (scoops == 3)
    System.out.print("Il gelato sta finendo");
else if (scoops > 9)
    System.out.print("Veloce che il gelato si scioglie!");
else if (scoops == 2)
    System.out.print("-2!");
else if (scoops == 1)
    System.out.print("-1!");
else if (scoops == 0)
    System.out.print("Andato!");
else
    System.out.print("C'è ancora molto gelato. Diamoci sotto!");
```



OPERATORE TERNARIO



SWITCH



```
switch (a) {  
    case 1:  
        // esegue se a è = 1  
        break;  
  
    case 2:  
        // esegue se a è = 2  
        break;  
  
    default  
        // esegue di default  
}
```

```
while (juggling) {  
    keepBallsInAir();  
}
```

```
while (condizione) {  
    // esegue finché condizione = true  
}
```

```
do {  
    // esegue finché condizione = true  
    // iniziando prima  
    // di entrare nel ciclo  
} while (condizione);
```

```
for (int i = 0; i < n; i++) {  
    // esegue n volte  
}
```

```
for (int i : list) {  
    // esegue tante volte  
    // quanto la lunghezza della lista  
}
```





Exercise

```
int count = 0;
```

```
for (int i = 0; i < 5; i++) {  
    count = count + i;  
}
```

Quanto vale count?



Exercise

```
int count = 0;
```

```
for (int i = 0; i < 5; i++) {  
    count = count + i;  
}
```

Quanto vale count?

$$0 + 1 + 3 + 6 + 4 = 10$$



Exercise

```
int tops = 5;

while (tops > 0) {
    for (int spins = 0; spins < 3; spins++) {
        System.out.print("La trottola sta girando");
    }

    tops--;
}
```

Quanti giri fanno in totale le trottole?





Exercise

```
int tops = 5;

while (tops > 0) {
    for (int spins = 0; spins < 3; spins++) {
        System.out.print("La trottola sta girando");
    }

    tops--;
}
```

Quanti giri fanno in totale le trottolo? $3 \times 5 = 15$



BE the compiler



```
class Exerciselb {  
    public static void main(String [] args) {  
        int x = 1;  
        while ( x < 10 ) {  
            if ( x > 3) {  
                System.out.println("big x");  
            }  
        }  
    }  
}
```

BE the compiler



```
class Exerciselb {  
    public static void main(String [] args) {  
        int x = 1;  
        while ( x < 10 ) {  
            if ( x > 3) {  
                System.out.println("big x");  
            }  
            }  
        }  
    }  
}
```

Manca l'incremento di x;
se x non supera 10, il
ciclo è infinito.

BE the compiler



```
class Exerciselb {  
    int x = 5;  
    while ( x > 1 ) {  
        x = x - 1;  
        if ( x < 3) {  
            System.out.println("small x");  
        }  
    }  
}
```

BE the compiler



```
class Exerciselb {  
    int x = 5;  
    while ( x > 1 ) {  
        x = x - 1;  
        if ( x < 3) {  
            System.out.println("small x");  
        }  
    }  
}
```

Manca il metodo!



- 1) Creare un progetto ElephantSong;
- 2) Implementare un prompt per chiedere il numero di elefanti;
- 3) Scrivere la cantilena degli elefanti con un numero di strofe pari al numero inserito dall'utente.

```
davide@MorningStar: ~/produzione/didattica/java/ElephantSong/dist
File Edit View Search Terminal Help

davide@MorningStar:~/produzione/didattica/java/ElephantSong/dist$ java -jar ElephantSong.jar
Quanti elefanti vuoi inserire? 7

1 elefante si dondolava sopra un filo di una ragnatela;
trovando la cosa molto interessante, va a chiamare un altro elefante...

2 elefanti si dondolavano sopra un filo di una ragnatela;
trovando la cosa molto interessante, vanno a chiamare un altro elefante...

3 elefanti si dondolavano sopra un filo di una ragnatela;
trovando la cosa molto interessante, vanno a chiamare un altro elefante...

4 elefanti si dondolavano sopra un filo di una ragnatela;
trovando la cosa molto interessante, vanno a chiamare un altro elefante...

5 elefanti si dondolavano sopra un filo di una ragnatela;
trovando la cosa molto interessante, vanno a chiamare un altro elefante...

6 elefanti si dondolavano sopra un filo di una ragnatela;
trovando la cosa molto interessante, vanno a chiamare un altro elefante...

7 elefanti si dondolavano sopra un filo di una ragnatela;
trovando la cosa molto interessante, vanno a chiamare un altro elefante...
```

byte

MSB

LSB

7

6

5

4

3

2

1

0

0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

+

23

bitwise operations

bitwise operations operate
one bit at a time

The results can be surprising
when you write them in base 10:

$$8 \& 3 = 0$$

but in binary it makes more
sense:

$$\begin{array}{r} 00001000 \leftarrow 8 \\ \& 00000011 \leftarrow 3 \\ \hline 00000000 \end{array}$$



Bitwise and: the
result is 1 if
BOTH bits are 1

$$1 \& 1 = 1$$

$$1 \& 0 = 0$$

$$0 \& 0 = 0$$

$$11 \& 10 = 10$$



Bitwise or: the
result is 1 if
EITHER bit is a 1

$$1 | 1 = 1$$

$$1 | 0 = 1$$

$$0 | 0 = 0$$

$$11 | 10 = 11$$



Bitwise xor: the
result is 1 if
EXACTLY ONE bit
is a 1

$$1 \wedge 1 = 0$$

$$1 \wedge 0 = 1$$

$$0 \wedge 0 = 0$$

$$11 \wedge 10 = 01$$



Bitwise not:
FLIP all the
bits

$$\sim 0 = 1$$

$$\sim 1 = 0$$

$$\sim 10 = 01$$



Left shift: add
0s to the end

$$1110 \ll 3 \\ = 1110000$$

$\ll n$ is like
multiplying by 2^n



Right shift: chop
bits off the end

$$01100001 \gg 2 \\ = 00011000$$

$\gg n$ is like
dividing by 2^n

there are actually two right shifts

unsigned right shift

$$253 \gg 1 = 126$$



always pad on the
left with a 0

signed right shift

$$-3 \gg 1 = -2$$



if the number is negative, pad on
the left with 1 instead of a 0

In some languages unsigned right shift is $\ggg$. In other languages, both
right shifts are $\gg$ and the integer's type determines which is used.